17 - Webové aplikace

Vrstvy webových aplikací a architektura MVC

- Současné webové aplikace jsou strukturovány do více vrstev, tradičně se uvádějí tři základní vrstvy:
    - **Prezentační vrstva** - zobrazení uživatelských výstupů v podobě webových stránek v prohlížeči.
    - **Logická vrstva -** dynamické generování webových stránek na straně webového serveru.
    - **Datová vrstva -** zdroj dat v podobě databázového serveru, datových souborů nebo internetových služeb.
- Architektura moderních webových aplikací je často založena na **návrhovém vzoru** (design pattern) označovaném zkratkou **MVC** (Model-View-Controller). Základní myšlenkou je **oddělení aplikační logiky od výstupu a dat**.
    - **Model** má za úkol podle zadaných parametrů zpracovat určitá vstupní data a připravit z nich data výstupní. Nerozhoduje o tom, jak bude s výstupními daty nakonec naloženo, jak budou zformátována a vypsána; model je zcela oddělen od výstupu. V případě použití *ORM* (objektově-relační mapování), korespondují modely přímo s databázovými tabulkami a představují třídy (např. *kniha*, *autor*, *vypujcka*), jejichž atributy se shodují s poli daných tabulek a metody většinou provádějí požadované operace s daty.
    - **View** (pohled) má na starost zobrazení výstupu dat, která mu byla předána. Může jít o pohled v podobě výpisu dat (sestavy), ale také o pohled s formulářem. Nejčastěji se k tvorbě pohledů používají tzv. **šablonovací systémy**, které umožňují do šablony, jejíž rámec tvoří HTML stránka, vkládat do symbolických bloků (často tvořených složenými závorkami) proměnné i různé příkazy (podmínky, cykly). Šablony se mohou vzájemně kombinovat a doplňovat - může např. existovat určitá výchozí šablona s layoutem stránek, kterou využívají a doplňují ostatní šablony.
    - **Controller** zprostředkovává komunikaci mezi klientem, modelem a pohledem. Každá entita mívá svůj vlastní controller - *UserController*, *ArticleController* atd.
- Tento základní návrhový vzor bývá v praxi používán i v různých modifikovaných podobách. Např. v případě frameworku Django se hovoří o architektuře **MVT** (Model-View-**Template**), nebo PHP framework Nette používá vzor **MVP** (Model-View-**Presenter**).

Typy webových API a jejich využití v praxi, websocket

- **API** (zkratka pro *Application Programming Interface*) označuje v informatice rozhraní pro programování aplikací. Jde o komplex procedur, funkcí, tříd nějaké knihovny (jiného programu, jádra OS), které může programátor využívat.
- API se nyní hojně využívá při **vývoji mobilních i webových aplikací** a tvorbě internetových stránek. Smyslem **webových API** je zajištění komunikace mezi dvěma platformami, které si vzájemně vyměňují data. V praxi se setkáme např. s využitím **API** z **Google Map** pro zobrazení polohy obchodu. API jsou využívána také čtečkami zpráv (**RSS**), kde umožňují výpis článků, nebo při přihlášení na webu a v mobilní aplikaci pomocí externího účtu (**Facebook**, **LinkedIn** atd.). Stejně tak umožňují přidávání komentářů například na sociální sítě. API může být součástí **účetního software** a díky tomu lze automaticky vygenerovat fakturu při nákupu z e-shopu a zapsat ji do databáze v účetnictví.
- K implementaci API lze přistupovat několika způsoby. Nejstarším prostředkem zůstává takzvaný *Simple Object Access Protocol*, zkráceně **SOAP**, určený pro volání procedur za využití **XML**. Jeho cílem je snazší **sdílení informací různých aplikací**, běžících v různých prostředích nebo v různých jazycích. Dnes je využíván **převážně bankami a pojišťovnami**; v ČR tento typ API využívá systém EET.
- **REST API** (někdy *RESTfull API*) je **architektura fungující na principu klient-server**, která ke své funkci využívá různé metody HTTP protokolu (např. metodu GET pro získání dat, POST pro vložení dat, PATCH pro aktualizaci, DELETE pro mazání objektů). Klient při dotazech potřebuje znát tzv. **endpointy** (koncové body - předepsané URL požadavky) serveru, na které zasílá dotazy (případně i data). **Komunikace** mezi klientem a serverem je **bezestavová**, server neukládá žádné informace a veškerou odpovědnost přenáší na klienta.
- Následníkem REST API je **GRAPHQL**, vyvinuté Facebookem. Jedná se o **jazyk dotazů**, který běží **na straně serveru**. Umožňuje stahovat data **z více zdrojů najednou** a nabízí i **větší rychlost zpracování** než REST API.
- **WebSocket** je nezávislý počítačový **komunikační protokol**, poskytující **plně duplexní (obousměrný) komunikační kanál přes jediné TCP připojení**. Používá se pro rychlý **real-time přenos dat** a vytváření aplikací v reálném čase, jako jsou **online chaty**, **kooperativní služby** (např. sdílená tabule, videokonference), **online hry** apod.

Uživatelské rozhraní webových aplikací

- Jedním z rysů moderních GUI je **oddělení obsahu od grafické úpravy**. To má pozitivní vliv především při větších úpravách aplikace, ale také při vytváření GUI aplikace pro jiné platformy (např. mobilní telefony).
- **Responzivní** (někdy *adaptivní*) **design** je způsob stylování HTML dokumentu, které zaručí, že zobrazení stránky bude optimalizováno pro všechny druhy nejrůznějších zařízení (mobily, netbooky, tablety atd.). Responzivní web design má tři základní úrovně:
    - **Flexibilní struktury** se dosahuje pomocí údajů zadávaných v procentech.
    - **Flexibilní obrázky** se přizpůsobují možnostem zobrazení díky tomu, že se šířka a výška obrázku neuvádějí uvnitř tagu <img>. Aby nedocházelo k přetékání, nastylují se všechny obrázky: img{max-width: 100%; height: auto;}.
    - **Media Queries** jsou pravidla, díky kterým lze měnit stylování dokumentu v závislosti na šířce obrazovky zobrazovaného zařízení. @media (max-width: 780px){...}
- K urychlení a usnadnění práce webdesignerů byly vytvořeny některé **responzivní frameworky**, např. Bootstrap.
- **Přístupnost webových stránek** (webové aplikace) znamená bezproblémového používání všemi skupinami uživatelů, včetně handicapovaných osob. Špatně navržené stránky či grafické rozhraní aplikace, které nerespektuje pravidla přístupnosti, může způsobit nedostupnost této služby pro lidi např. se zrakovým postižením (*Blind Friendly Web*), uživatele s pomalejším připojením, seniory, apod. **Podle zákona musejí všechny weby státní správy projít auditem přístupnosti.**

Formuláře, formulářové prvky a validace vstupních dat

- **Formuláře** slouží k zadávání dat; používají se k přihlašování uživatelů, pro vyhledávání, pro uživatelské komentáře, k vyplňování zákaznických údajů v e-shopech atd. Data mohou být z formuláře odeslána na server, kde jsou zpracována pomocí serverových aplikací a uložena do databáze nebo souboru, odeslána mailem apod.
- Základním elementem webového formuláře je značka: *<form action="skript.php" method="post">...</form>*
  Atribut **action** obsahuje URL adresu skriptu, jemuž budou odeslána data z formuláře. Atribut **method** rozhoduje o způsobu odeslání dat. Výchozí metodou **GET** (nemusí být zadána) jsou data odeslána jako viditelná součást URL požadavku a jejich délka by neměla přesáhnout určitý počet znaků (obvykle kolem 2000). I proto se doporučuje využívat metodu **POST**, kdy jsou data posílána v těle požadavku, nezobrazují se v URL adrese a nejsou omezena velikostí.
- Běžný webový formulář je tvořen popisky (značka *<label>*) a formulářovými prvky k vyplňování hodnot. Nejčastěji se setkáváme s elementem <input>; atribut **type** v tomto případě určuje charakter dat (např. *text*, *number*, *date*, *password*, *file*). Typ *checkbox* představuje zaškrtávací políčko, typ *radio* se používá pro množinu přepínačů. Existuje i typ *hidden*, který se dá využít k vložení skrytých hodnot. Pomocí značky *<select>* vytváříme seznamy s nabídkou předvoleb, párová značka *<textarea>* umožňuje editaci víceřádkového textu. Tlačítka slouží k potvrzování akcí nebo odesílání dat (submit).
- Tvůrce webové aplikace by měl vždy zajistit, aby hodnoty zadávané do formuláře byly vždy pečlivě zkontrolovány buď ještě před jejich odesláním **na straně klienta** (pomocí vhodných atributů HTML nebo JavaScriptu), v každém případě však **na straně serveru**. **Validací formulářových dat** předcházíme možným bezpečnostním rizikům, které mohou způsobit pokusy záměrně vložit podvržená data a způsobit problémy aplikaci (např. Cross site scripting, SQL injection), nebo i nechtěná chybná zadání uživatelů, která mohou mít neblahý vliv na data v databázi (porušení integrity).
- K validaci zadaných dat můžeme jak na straně klienta, tak i na straně serveru využít (mimo jiné) tzv. **regulárních výrazů**. Regulární výraz (*regular expression*) je speciální řetězec znaků, který představuje určitý vzor (masku) pro textové řetězce. Většina moderních programovacích jazyků používá **Perl-compatible regulární výrazy**, které mohou být sestaveny pomocí speciálních symbolů: *\^\d{3} ?\d{2}$* (regulární výraz pro kontrolu PSČ). K formulářovému prvku lze regulární výraz vložit do atributu pattern.

Metody autentizace a autorizace na webu

- **Autentizace** znamená **ověření identity** nějakého subjektu (člověka, osoby, systému). V rámci webových aplikací se v současné době velmi často používá tzv. **vícefaktorová autentizace**, kdy se běžné ověření identity pomocí uživatelského jména a hesla kombinuje s dalšími metodami (např. zaslání PINu na mobilní telefon).
- Nejčastěji se ve webových aplikacích setkáváme s autentizací, která využívá HTML formuláře a **session proměnné**. Uživatel se přihlašuje prostřednictvím přihlašovacího formuláře, na němž vyplňuje minimálně uživatelské jméno a heslo.

Po odeslání formuláře jsou zadané uživatelské údaje porovnávány s odpovídajícími daty, jež bývají uloženy zpravidla v databázi. Aby nemuselo docházet k neustálému ověřování identity uživatele během práce s aplikací, jsou ověřené identifikační údaje uchovávány v session proměnných na serveru. Pro odhlášení pak stačí zrušit session proměnnou (**session id**); v případě automatického odhlášení se při každém požadavku porovnává aktuální čas s časem posledního přístupu v session proměnné.

- Aplikace by v žádném případě neměla ukládat hesla v odkryté podobě, ale využívat některou ze šifrovacích metod - nejčastěji hašovací funkci (např. md5, sha atd.). S ohledem na existenci různých slovníků s velkou zásobou shromážděných otisků pro častěji používaná hesla je bezpečnější použít na straně aplikačního serveru tzv. **saltování**, kdy se k zadanému řetězci přidá "sůl" v podobě přidaných znaků (tato "sůl" je pro daný server vždy stejná).
- **Klientské certifikáty**. Tato autentizační metoda využívá mechanismus speciálních vrstev pro šifrování - SSL/TLS. Předpokladem je digitální certifikát v podobě veřejného šifrovacího klíče, vydaného certifikační autoritou. Certifikátem se prokazuje nejen server, ale i uživatel, což může být pro neznalého uživatele komplikovanější. Klientské certifikáty se hojně využívaly např. v internetovém bankovnictví, dnes už jsou na ústupu.
- **OpenID** je otevřený standard pro autentizaci uživatelů, který odstraňuje potřebu na straně provozovatele služby poskytovat a vyvíjet vlastní systémy pro autentizaci. OpenID má tvar unikátního URL, ke kterému je přiřazeno heslo. Služba, která uživatelům autentizaci pomocí OpenID nabízí, při přihlašování uživatele přesměruje požadavek na ověření identity na správce daného OpenID účtu. Ten vrátí informaci o povolení či zamítnutí žádosti o autentizaci. Autentizace pomocí OpenID je v ČR poskytována pod názvem *mojeID* správcem národní domény sdružením CZ.NIC.
- **OAuth** je otevřený protokol, který umožňuje sdílet určitá uživatelská data (případně identity) uložené na jednom serveru s jiným serverem, aniž by uživatelé museli vyzradit své přístupové údaje. Uživatel se přihlašuje pouze na straně tzv. **service providera** (obvykle velké služby typu Facebook, Github) a zde povolí přístup pro externí aplikaci, která je označovaná jako **consumer**. Když tato externí aplikace získala povolení k určitému přístupu (**access token**), může využívat některé chráněné zdroje - kupříkladu vypisovat všechny přátele uživatele z dané sociální sítě apod.
- **Autorizace** je proces **získání přístupu k částem aplikace** (systému) a vykonat určitou činnost. Na počátku tohoto procesu je obvykle *autentizace uživatele*, zjištění jejich **rolí** a souvisejících **práv**. Na základě toho mohou být uživatelům přidělena potřebná oprávnění, nebo naopak odepřen přístup ke konkrétním informacím nebo funkcím. V návaznosti na zvolenou bezpečnostní strategii bývají oprávnění řešena na různých úrovních a rozdělena mezi více subjektů – nejčastěji mezi administrátory, bezpečnostní správce a běžné uživatele.

Redakční systémy

- Zkratkou **CMS** (Content Management System = systémy pro správu obsahu) je označován software, pro který se vžil častěji používaný název **redakční systém**. Jedná se zpravidla o **komplexní webovou aplikaci** používanou pro tvorbu a úpravu obsahu webu bez znalostí kódování.
- CMS umožňují vkládání různých článků a příloh (včetně tabulek a obrázků) do předem určených kategorií, případně úpravu všech textů na webu. Používají se k tomu tzv. **WYSIWYG editory**, jejichž ovládání se velmi podobá ovládání běžných desktopových textových procesorů (např. MS Word), které fungují v internetových prohlížečích a v pozadí generují HTML kód, například *FCKEditor* a *TinyMCE* (vytvořené v jazyce JS).
- Častou funkcí CMS je i správa obrázků a celých fotogalerií, většina redakčních systémů obsahuje i moduly pro správu multimediálního obsahu, včetně audio a video souborů. Výjimkou není ani možnost spravovat diskuse a komentáře. Vyspělé CMS dokáží přímo manipulovat se strukturou webu a s využitím různých přídavných modulů přizpůsobovat funkce redakčního systému konečnému záměru - s jejich pomocí je tak možné vytvářet weblogy, firemní stránky, elektronické obchody, zpravodajské portály atd.
- Nejpopulárnějším redakčním systémem je **WordPress** - open source redakční systém napsaný v jazyce PHP a využívající databázový systém MySQL. Původně byl vyvinut jako blogovací systé, ale díky rozsáhlé komunitě programátorů a existenci mnoha rozšíření je dnes základem nejrůznějších webových projektů. Jednou z jeho výhod je snadná instalace jádra systému, po níž může následovat instalace různých doplňků (pluginů) i šablon grafického vzhledu (témat).
- Redakční systém je spravován prostřednictvím **administrační části aplikace**, do níž mají přístup uživatelé podle nadefinovaných oprávnění.
- Nevýhodou Wordpressu, ale i jiných rozšířených CMS je množství útoků, které využívají slabých míst těchto systémů k jejich ovládnutí, poškození nebo i zneužití k jiné kybernetické kriminalitě. Při nasazení jakéhokoliv redakčního systému je proto nezbytné důsledně dodržovat všechny bezpečnostní zásady, zejména pravidelně aktualizovat jádro systému, ale i pluginy.